

JAVA COLLECTION FRAMEWORK

1.COLLECTION FRAMEWORK FOUNDATION

1. Introduction to Collections

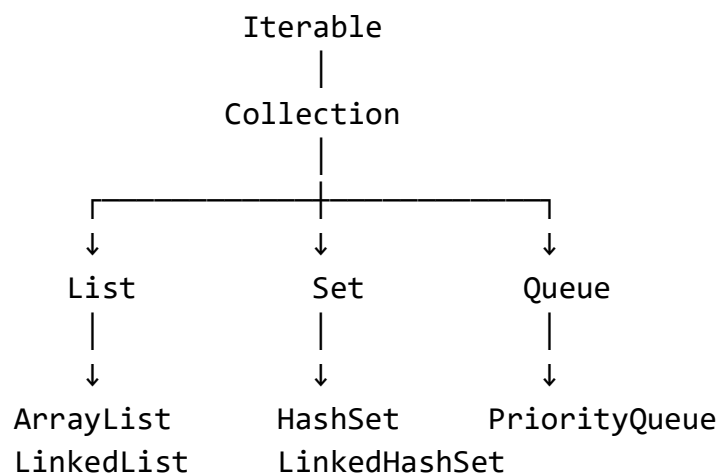
- What is a Collection?
- What is Collection Framework?
- Why do we need Collections?
- Advantages of Collection Framework
- Real-life examples

2. Arrays vs Collections

- Limitations of arrays
- Fixed size vs dynamic size
- Arrays vs Collections
- When to use arrays
- When to use collections

3. Collection Framework Hierarchy

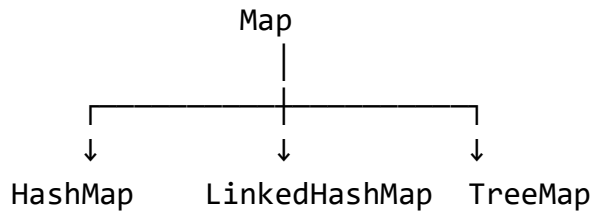
Understand the basic structure:



Vector
Stack

TreeSet

And separately:



Very important: Map is part of the Collection Framework, but Map does **not** extend the Collection interface.

4. Important Interfaces

- Iterable
- Collection
- List
- Set
- Queue
- Map

5. Generics

Before working seriously with collections, understand:

```
ArrayList<String> names;
```

What does <String> mean?

- Generics
- Type safety
- Why generics are useful
- Basic generic syntax

2. THE MAIN COLLECTIONS

1. LIST

Concept

A List is used when:

- Order matters
- Duplicates are allowed
- Elements are accessed using indexes

Example:

```
10  
20  
20  
30
```

Duplicates

A. ArrayList

- Creating ArrayList
- Adding elements
- Accessing elements
- Updating elements
- Removing elements
- Searching elements
- Iterating
- Size
- Common methods
- Advantages/disadvantages

Example:

```
ArrayList<String> names = new ArrayList<>();
```

B. LinkedList

- Concept
- Basic working
- Adding/removing elements
- Common methods
- ArrayList vs LinkedList

C. Vector

- Concept
- Features
- Basic methods
- Vector vs ArrayList

D. Stack

- LIFO concept
- push()
- pop()
- peek()
- empty()

Remember:

Stack

↓

LIFO

↓

Last In → First Out

2. SET

Set is used when you generally want:

Unique elements

Example:

10
20
10
30

becomes conceptually:

10
20
30

A. HashSet

- Unique elements
- No guaranteed ordering
- Basic methods
- Adding/removing/searching

B. LinkedHashSet

- Unique elements
- Maintains insertion order

C. TreeSet

- Unique elements
- Sorted order

Very Important Comparison

HashSet

↓

Unique

↓

No guaranteed insertion order

LinkedHashSet

↓

Unique



Insertion order

TreeSet



Unique



Sorted order

3. QUEUE

Queue follows:

FIFO — First In, First Out

Think about a line at a ticket counter:

Person 1 → Person 2 → Person 3



First person leaves first

Topics

- Queue concept
- FIFO
- add()
- offer()
- remove()
- poll()
- peek()
- element()

PriorityQueue

- Priority-based processing
- Basic operations

- Difference between Queue and PriorityQueue

4. MAP

This is extremely important.

Map stores data in:

KEY → VALUE

Example:

101 → Regina

102 → Priya

103 → Anjali

Think of it like a dictionary:

WORD → MEANING

A. HashMap

- Key-value pairs
- Keys must be unique
- Values can be duplicated
- Basic methods
 - put()
 - get()
 - remove()
 - containsKey()
 - containsValue()

Example:

```
HashMap<Integer, String> students = new HashMap<>();
```

```
students.put(101, "Regina");
```

```
students.put(102, "Priya");
```

B. LinkedHashMap

- Key-value pairs
- Maintains insertion order

C. TreeMap

- Key-value pairs
- Keys are sorted

Map Comparison

HashMap

↓

Fast general-purpose map

↓

No guaranteed insertion order

LinkedHashMap

↓

Maintains insertion order

TreeMap

↓

Sorted by keys

3. COLLECTIONS IN REAL JAVA PROGRAMMING

1. Iterating Through Collections

Learn:

Enhanced for loop

```
for(String name : names) {  
    System.out.println(name);  
}
```

Iterator

```
Iterator<String> it = names.iterator();
```

Learn:

- hasNext()
- next()
- remove()

ListIterator

- Forward traversal
- Backward traversal
- hasPrevious()
- previous()

2. Collections Class

Java provides a utility class called:

Collections

Important methods:

- sort()
- reverse()
- shuffle()
- max()
- min()
- frequency()
- swap()

Example:

```
Collections.sort(numbers);
```

3. Comparable

Used when you want to define the **natural/default ordering** of objects.

Important method:

```
compareTo()
```

Concept:

```
Objects
  ↓
Comparable
  ↓
compareTo()
  ↓
Natural ordering
```

4. Comparator

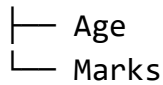
Used when you want to define a **custom ordering**.

Important method:

```
compare()
```

For example:

```
Students
  ↓
Sort by:
  |— Name
```



You can create different sorting rules.

Comparable vs Comparator

Comparable	Comparator
<code>compareTo()</code>	<code>compare()</code>
Natural ordering	Custom ordering
Usually defined inside the class	Separate comparison logic
One main ordering	Can define multiple orderings

Memory trick

Comparable = "I compare myself."

Comparator = "Someone else compares the objects."

5. Important Collection Methods

You should become comfortable with:

```
add()
addAll()
remove()
removeAll()
contains()
containsAll()
size()
isEmpty()
clear()
get()
```

`set()`
`indexOf()`

Don't memorize them randomly. Learn them according to the collection you're using.

6. Exception Handling with Collections

Basic understanding of:

- `ConcurrentModificationException`
- `IndexOutOfBoundsException`
- `NoSuchElementException`